

ONEDRAFT

CAD-AI — Aria Powered

AI Evaluation, Guardrails & Deterministic Command Verification Specification

Version 1.0

Document Status: System Architecture Specification

System: OneDraft / CAD-AI

Engine: Aria Powered

Primary Output: Verified Architectural Project Operations

Drawing Standard: ARCH D / jurisdiction-configurable

Model Principle: Deterministic Project State / AI-Assisted Generation

1. PURPOSE

OneDraft shall establish the AI Evaluation, Guardrails & Deterministic Command Verification subsystem governing the transition between Aria-generated intelligence and executable changes to the OneDraft Project Model.

The purpose of this subsystem is to ensure that AI-generated architectural interpretations, commands, modifications, redline instructions and project operations are evaluated before they are permitted to modify authoritative project state.

Aria shall operate as the intelligence and generative reasoning layer.

The OneDraft verification architecture shall operate as the deterministic control layer.

Accordingly, the fundamental architecture is:

AI OUTPUT → STRUCTURED COMMAND → VERIFICATION → PROJECT MODEL

and not:

AI OUTPUT → DIRECT PROJECT MUTATION

The system must therefore distinguish between an AI-generated proposal and an authoritative project-state transition.

No AI-generated command shall directly modify the authoritative Project Model without passing through the applicable schema, authorization, project-state, constraint, geometry, compliance, coordination and execution-policy controls.

2. AI VERIFICATION IDENTITY

Every AI-generated executable operation shall receive a unique immutable `Command_ID`.

The command verification identity record shall contain:

Command_ID
Project_ID
Organization_ID
User_ID
Revision_ID
Model_Version
AI_Provider
AI_Model
AI_Model_Version
Prompt_Version
Context_Version
Command_Schema_Version
Guardrail_Version
Verifier_Version
Confidence_Assessment
Risk_Classification
Verification_Status
Execution_Decision
Created_Date
Verified_Date
Executed_Date

The `Command_ID` shall remain associated with the complete lifecycle of the AI-generated operation.

Verification shall create a traceable relationship between the original AI output, the structured command, the verification result and any resulting Project Model revision.

3. AI OUTPUT CLASSIFICATION

The system shall classify every substantive Aria output before determining whether execution is permitted.

The classification structure shall support, at minimum:

Proposal
Command
Question
Explanation

Warning

Rejection

Requires Review

A **Proposal** shall represent an AI-generated recommendation that has not yet been authorized for execution.

A **Command** shall represent a structured operation capable of being evaluated by the OneDraft verification architecture.

A **Question** shall identify information required before the requested operation can be reliably interpreted.

An **Explanation** shall contain informational output without Project Model mutation authority.

A **Warning** shall identify a condition requiring attention.

A **Rejection** shall identify an operation that cannot proceed under the applicable system controls.

A **Requires Review** state shall identify an operation that may be technically possible but requires human, customer or professional review before execution.

AI output classification shall therefore prevent natural-language generation from being treated as an implicit database instruction.

4. STRUCTURED COMMAND MODEL

Aria-generated executable operations shall be converted into structured OneDraft commands before they are submitted to the execution architecture.

A structured command shall identify, where applicable:

Operation

Project_ID

Revision_ID

Target_Objects

Parameters

Affected_Constraints

Required_Preservation_Rules

Reason

Confidence

Risk_Classification

Source_Reference

The structured command shall provide a deterministic representation of the intended Project Model operation.

Natural-language interpretation alone shall not constitute an executable operation.

The command schema shall therefore provide a formal boundary between Aria's generative reasoning and the transactional application services responsible for changing project state.

5. COMMAND SCHEMA VALIDATION

Every structured AI command shall undergo schema validation before further processing.

The validation process shall verify:

Required fields

Field types

Enumerated values

Identifier validity

Parameter structures

Numeric ranges

Object references

Operation compatibility

Revision identifiers

Command schema version

Malformed commands shall be rejected before they reach Project Model mutation services.

Schema validity shall not constitute authorization or architectural correctness.

A command may therefore be structurally valid while subsequently failing authorization, constraint, geometry, compliance or coordination verification.

6. AUTHORIZATION VERIFICATION

AI-generated commands shall remain subject to the Identity, Access & Security architecture established in the preceding OneDraft stacks.

The verification system shall evaluate:

User_Identity

Organization_Membership

Project_Membership

Project_Role

Command_Privilege

Resource_Ownership

Administrative_Restrictions

System_Privilege

A valid command shall not automatically be considered an authorized command.

Accordingly:

VALID COMMAND $\neq$ AUTHORIZED COMMAND

Authorization shall be determined through the established OneDraft access-control architecture rather than through the AI model itself.

Aria shall have no implicit authority to bypass OneDraft authorization boundaries.

7. PROJECT STATE VERIFICATION

Before execution, the command shall be evaluated against the current authoritative Project Model state.

The system shall verify:

Project exists

Project is active

Referenced revision exists

Referenced model version exists

Target objects exist

Target objects belong to the project

Target objects are available for modification

Required project state is compatible with the operation

The command shall not be permitted to modify a project state that no longer corresponds to the state against which the AI operation was generated.

Stale project context shall therefore be treated as a verification condition rather than silently reconciled against current state.

8. REVISION CONTROL

Every AI-generated command shall reference the project revision against which it was generated.

The system shall compare the command's expected revision with the current authoritative revision.

For example:

Expected Revision = 14

Current Revision = 14

The command may proceed to subsequent verification.

Where:

Expected Revision = 14

Current Revision = 15

the command shall not overwrite or silently modify Revision 15.

The system shall return a revision conflict and require the operation to be regenerated, reconciled or otherwise resolved through the applicable project workflow.

Historical project revisions shall remain immutable.

9. TARGET OBJECT VERIFICATION

Every AI-generated command affecting the Project Model shall identify the objects upon which the operation is intended to act.

The verification system shall determine whether:

The target object exists

The target object belongs to the project

The target object supports the requested operation

The target object is editable

The target object is not deleted

The target object is not superseded

The target object is not locked against the operation

The target object maintains valid relationships with the Project Model

An operation shall not be permitted to act upon an object that does not satisfy the applicable target conditions.

10. PARAMETER VERIFICATION

AI-generated parameters shall be independently validated before they are applied to the Project Model.

Parameter verification shall support architectural values including:

Dimensions

Coordinates

Elevations

Angles

Offsets

Clearances

Wall Thicknesses

Door Widths

Room Areas

Slopes
Heights
Grid Positions
Spacing
Rotation
Scale

The AI model may propose parameter values.

The deterministic OneDraft systems shall determine whether those values are valid within the current Project Model, applicable constraints and configured project rules.

AI confidence shall not substitute for parameter validation.

11. CONSTRAINT PRESERVATION

Constraint preservation shall constitute a primary verification function of the subsystem.

The system shall identify constraints affected by an AI-generated operation.

Constraints may include:

Property Boundaries
Setbacks
Structural Grids
Structural Members
Fire Separations
Egress Paths
Accessible Routes
Minimum Clearances
Floor-to-Floor Heights
Shaft Locations
Mechanical Zones
Plumbing Stacks
Existing Conditions
Survey Coordinates
Client-Locked Dimensions
Jurisdictional Restrictions
Approved Redlines

The system shall determine whether execution of the proposed command would preserve, modify, violate, delete or ambiguously affect the applicable constraints.

12. CONSTRAINT DECISION

The constraint verification system shall support, at minimum:

Preserved

Modified with Authorization

Violated

Ambiguous

A preserved constraint shall permit continued verification.

A modified constraint shall require the applicable authorization and state transition.

A violated constraint shall cause the command to be rejected unless the governing workflow explicitly permits the constraint itself to be revised.

An ambiguous constraint shall cause the command to require review or additional information.

Ambiguity shall never be interpreted as authorization.

13. GEOMETRIC VERIFICATION

Any command affecting architectural geometry shall be evaluated through the computational geometry architecture established in the preceding OneDraft stacks.

Geometric verification shall support:

Topological Validity

Intersection Detection

Collision Detection

Clearance

Connectivity

Containment

Alignment

Dimension Consistency

Closed-Loop Geometry

Room Boundary Integrity

Wall Connectivity

Opening Containment

Level Association

Coordinate Consistency

The AI model shall not constitute the authoritative source of geometric correctness.

The OneDraft geometry engine shall remain authoritative for deterministic geometric computation.

14. REGULATORY VERIFICATION

Where an AI-generated command affects a regulated project condition, the command shall be evaluated through the Compliance & Regulatory Engine and Validation & Verification Engine established in the preceding architecture.

The regulatory verification sequence shall remain:

**CODE MANIFEST → CODE RULES → APPLICABILITY → PROJECT MODEL →
COMMAND EFFECT → VALIDATION**

The AI model shall not independently create or override the authoritative regulatory baseline.

The applicable jurisdictional code manifest shall remain associated with the project state against which the command is evaluated.

15. COORDINATION VERIFICATION

AI-generated commands shall be evaluated for effects upon coordinated project systems where applicable.

The coordination architecture shall support:

Architectural
Structural
Mechanical
Electrical
Plumbing
Life Safety
Accessibility
Site
Civil
Vertical Circulation
Fire Protection

A modification to one project element may affect multiple dependent elements.

The system shall therefore evaluate downstream effects where such dependencies are represented within the Project Model.

An unresolved coordination conflict shall prevent the command from being represented as a fully verified project-state transition.

16. AI CONFIDENCE

Aria may provide a confidence assessment associated with a generated interpretation or command.

Confidence shall be treated as an AI assessment rather than a deterministic proof of correctness.

A confidence value shall not represent:

- Legal correctness
- Structural correctness
- Regulatory approval
- Municipal approval
- Professional certification
- Building-permit approval

Confidence shall instead represent the AI system's assessment of its own generated interpretation under the applicable model and context.

Deterministic OneDraft verification shall remain authoritative.

17. CONFIDENCE POLICY

The system shall support configurable confidence thresholds.

A high-confidence command may proceed toward execution where all mandatory deterministic checks pass.

A medium-confidence command may require additional verification or review.

A low-confidence command may be rejected or routed for review.

The system shall enforce the following principle:

HIGH AI CONFIDENCE + DETERMINISTIC FAILURE = REJECT

AI confidence shall never override:

- Constraint failure
- Geometry failure
- Compliance failure
- Authorization failure
- Revision conflict
- Coordination failure
- Schema failure

18. GUARDRAIL ARCHITECTURE

The OneDraft guardrail architecture shall operate across multiple control boundaries.

The system shall support:

Input Guardrails

Context Guardrails

Output Guardrails

Command Guardrails

Execution Guardrails

Compliance Guardrails

Audit Guardrails

Input guardrails shall identify incomplete, conflicting or ambiguous project instructions.

Context guardrails shall ensure that Aria receives project information corresponding to the applicable project state.

Output guardrails shall evaluate generated responses before they are transformed into executable operations.

Command guardrails shall control structured commands immediately before deterministic execution.

Execution guardrails shall determine whether the verified command is eligible for autonomous execution.

Audit guardrails shall ensure that the resulting operation remains traceable.

19. INPUT GUARDRAILS

Input guardrails shall identify conditions that may prevent reliable interpretation of the customer's instructions.

The system shall identify:

Missing Dimensions

Conflicting Specifications

Ambiguous Object References

Missing Project State

Unsupported Operations

Incomplete Requirements

Conflicting Constraints

Unresolved Redlines

The system shall distinguish between missing information and negative information.

Missing information shall not be interpreted as permission to invent project requirements.

Where necessary, Aria shall request additional project information before generating an executable command.

20. CONTEXT GUARDRAILS

The context supplied to Aria shall be bounded by the authoritative project information available to the current command.

Context may include:

- Current Project Revision
- Current Model Version
- Project Requirements
- Project Constraints
- Project Geometry
- Code Manifest
- Validation Results
- Active Redlines
- Approved Changes
- Locked Objects
- Customer Instructions

Stale project information shall not be silently represented as current project state.

Context construction shall remain coordinated with the AI Intelligence & Model Provider Architecture established in the preceding stack.

21. OUTPUT GUARDRAILS

Aria output shall be evaluated before it is accepted as a structured command.

The system shall identify:

- Malformed Output
- Unsupported Operations
- Missing Parameters
- Contradictory Instructions
- Unsupported Assumptions
- Unresolved Ambiguity
- Invalid Object References
- Unsafe State Transitions

Output that cannot be deterministically interpreted shall not proceed to Project Model execution.

22. COMMAND GUARDRAILS

Command guardrails shall operate immediately before Project Model mutation.

The command guardrail layer shall verify:

Command Identity

Authorization

Project Revision

Target Objects

Parameters

Constraints

Dependencies

Operation Policy

Execution Policy

The command shall not proceed to transactional mutation until the required verification conditions have been satisfied.

23. EXECUTION DECISION

Every AI-generated command shall receive an explicit execution decision.

The decision model shall support:

Execute

Reject

Require Review

Retry

Regenerate

Defer

Execute shall indicate that all mandatory verification conditions have passed.

Reject shall indicate that the command cannot proceed.

Require Review shall indicate that the operation requires an additional decision.

Retry shall indicate a recoverable processing condition.

Regenerate shall indicate that the AI output itself is insufficient for reliable execution.

Defer shall indicate that execution depends upon another project state transition.

24. DETERMINISTIC COMMAND VERIFICATION

The command verification system shall evaluate every executable AI command against deterministic project information and versioned verification rules.

The verification function shall conceptually evaluate:

Project State

Command

Authorization

Constraints

Code Manifest

Validation Rules

Coordination State

Execution Policy

The verification result shall provide a deterministic decision for a fixed set of inputs, rules and verifier versions.

The verification engine shall therefore create a formal boundary between probabilistic AI generation and deterministic project execution.

25. VERIFICATION RESULT

Each completed verification shall produce a structured verification result.

The result shall contain, at minimum:

Verification_ID

Project_ID

Revision_ID

Command_ID

Decision

Command_Validity

Authorization_Status

Revision_Status

Constraint_Results

Geometry_Results

Compliance_Results

Coordination_Results

Confidence_Assessment
Risk_Classification
Warnings
Failures
Verifier_Version

The verification result shall remain associated with the command throughout its lifecycle.

26. FAILURE CLASSIFICATION

Verification failures shall be classified rather than returned as an undifferentiated error.

The failure structure shall support:

Schema Error
Authorization Error
Revision Conflict
Project State Error
Target Object Error
Parameter Error
Constraint Violation
Geometry Error
Compliance Error
Coordination Error
Confidence Error
Policy Error
Ambiguity Error
Dependency Error
System Error

Failure classification shall permit deterministic handling, reporting, telemetry and audit.

27. GUARDRAIL VERSIONING

Guardrails shall be version controlled.

Each verification record shall identify the applicable:

Guardrail_Version
Verifier_Version
Command_Schema_Version

Project_Model_Version
Code_Manifest_Version

Changes to verification logic shall therefore remain distinguishable across project history.

Historical verification records shall not silently acquire the semantics of later guardrail versions.

28. VERIFICATION PROVENANCE

The system shall retain provenance sufficient to establish how an AI-generated command was evaluated.

Verification provenance shall connect:

AI Output → Command → Project State → Rules → Verification → Decision → Execution

Where available through the AI architecture, the provenance record shall additionally identify:

AI Provider
AI Model
AI Model Version
Request Identifier
Prompt Version
Context Version
Generation Timestamp

This provenance shall integrate with the Data Lifecycle, Retention & Provenance System established in the preceding architecture.

29. IMMUTABLE VERIFICATION RECORD

Completed verification records shall be retained as immutable project-history records where required by the applicable retention policy.

The record shall identify:

Original AI Output Reference
Structured Command
Project Revision
Model Version
Constraints Evaluated
Rules Evaluated
Verification Results
Execution Decision

Execution Result
Verifier Version
Timestamp
Actor

The verification record shall not be modified merely because a later project revision supersedes the command.

30. DETERMINISTIC REPLAY

The system shall support deterministic replay of verification decisions where the required historical inputs remain available.

Replay shall use the applicable:

Project Revision
Project Model Version
Command
Constraints
Code Manifest
Verifier Version
Command Schema
Guardrail Version

The purpose of replay is to establish whether the same deterministic verification architecture produces the same result from the same authoritative inputs.

31. AI AND DETERMINISTIC SYSTEM BOUNDARY

The OneDraft architecture shall explicitly distinguish between the probabilistic AI layer and the deterministic project-control layer.

The AI layer shall provide:

Interpretation
Generation
Reasoning
Proposal
Command Construction

The deterministic layer shall provide:

Schema Validation
Authorization
State Validation
Constraint Verification
Geometry Verification
Compliance Verification
Coordination Verification
Execution Policy
Transactional Mutation
Audit

The system shall not attempt to make the AI model itself the authoritative deterministic system.
Instead, OneDraft shall make the transition from AI output to authoritative Project Model state controlled and verifiable.

32. HUMAN REVIEW ESCALATION

The system shall support escalation where an AI-generated operation cannot safely proceed under autonomous execution policy.

Escalation conditions may include:

Insufficient Confidence
Ambiguous Requirements
Multiple Valid Interpretations
Constraint Conflicts
Unresolved Coordination
Locked Project Conditions
High-Risk Operations
Required Professional Review

The resulting review decision shall become part of the project revision history where it affects Project Model state.

33. CUSTOMER REVIEW

Customer review shall remain a defined stage within the OneDraft project lifecycle.

Customers shall be able to:

Review
Redline

Modify Instructions
Reject Proposals
Request Regeneration
Approve Permitted Operations

Customer approval shall remain distinguishable from automated deterministic verification.

The system shall preserve the relationship between customer instruction, AI interpretation, system verification and resulting project state.

34. REDLINE VERIFICATION

Redlines processed through Aria shall enter the same command-verification architecture as other AI-generated operations.

The redline workflow shall therefore be:

**REDLINE → ARIA INTERPRETATION → STRUCTURED COMMAND → VERIFICATION
→ PROJECT MODEL → DOCUMENTATION**

A redline shall not automatically constitute permission for an unverified Project Model mutation.

The resulting revision shall remain traceable to the originating redline.

35. AUTONOMOUS EXECUTION

OneDraft may permit autonomous execution for operations falling within a configured execution policy.

Operations may include, where permitted:

Dimension Adjustments
Annotation Placement
Sheet Composition
Graphic Corrections
Alignment Operations
Standardized Drafting Refinements
User-Authorized Spatial Adjustments

Execution policy shall be configurable according to project state, command type, customer settings and applicable review requirements.

High-impact operations may require explicit review before execution.

36. COMMAND RISK CLASSIFICATION

AI-generated commands shall support risk classification.

The minimum classification structure shall support:

Class A — Low Impact

Class B — Moderate Impact

Class C — High Impact

Class D — Review Required

Risk classification may consider:

Affected Object Type

Number of Affected Objects

Structural Relevance

Life-Safety Relevance

Code Relevance

Geometry Complexity

Coordination Dependencies

Project Stage

Customer Policy

Professional Review Requirements

Risk classification shall supplement, and shall not replace, deterministic validation.

37. CASCADE IMPACT ANALYSIS

The verification system shall identify downstream project effects where the Project Model contains the required relationships.

For example:

**WALL MODIFICATION → ROOM BOUNDARY → DOOR LOCATION → EGRESS →
ACCESSIBILITY → MECHANICAL ROUTING → DOCUMENTATION**

A command shall therefore be evaluated not only against its direct target objects but also against affected dependent objects where such relationships are represented in the Project Model.

38. SAFE PROJECT STATE TRANSITION

An AI-generated command shall be accepted only where the resulting Project Model state can be established as a valid state transition.

The fundamental transaction shall be:

CURRENT PROJECT STATE + VALIDATED COMMAND → VALID NEXT PROJECT STATE

Where the proposed state cannot satisfy the required verification conditions:

CURRENT PROJECT STATE + INVALID COMMAND → REJECT

The authoritative current project state shall remain unchanged following a rejected operation.

39. TRANSACTIONAL EXECUTION

Verification and Project Model mutation shall remain logically distinct while being coordinated through the transactional application architecture.

The intended execution sequence shall be:

VERIFY → BEGIN TRANSACTION → RECHECK REVISION → APPLY MUTATION → CREATE MODEL VERSION → CREATE REVISION → CREATE CHANGE RECORDS → UPDATE CURRENT STATE → WRITE AUDIT EVENT → COMMIT

Where execution fails before commit, the transaction shall be rolled back.

The system shall not permit a partially committed AI-generated architectural operation to become authoritative Project Model state.

40. IDEMPOTENCY AND RETRY SAFETY

AI commands shall support idempotent execution where applicable.

The system shall support:

Command_ID

Request_ID

Idempotency_Key

Repeated requests shall not unintentionally produce duplicate Project Model mutations.

Retry behavior shall distinguish between an unsuccessful AI generation request and a Project Model transaction whose outcome has already been committed.

A committed operation shall not be repeated without first determining its authoritative transaction state.

41. OBSERVABILITY

The subsystem shall integrate with the Observability, Telemetry & System Health architecture established in the preceding operations architecture.

The system shall emit events including:

- AI Command Received
- AI Command Parsed
- AI Command Rejected
- AI Command Verified
- AI Command Executed
- AI Command Escalated
- Constraint Check Failed
- Geometry Check Failed
- Compliance Check Failed
- Coordination Check Failed
- Revision Conflict
- Verification Completed

These events shall permit operational monitoring of AI command quality, verification failures and execution behavior.

42. TESTING AND QUALITY ASSURANCE

The subsystem shall integrate with the Testing, Quality Assurance & Verification System established in the preceding architecture.

Testing shall include:

- Unit Testing
- Integration Testing
- Contract Testing
- Command Schema Testing
- Guardrail Testing
- Constraint Testing
- Geometry Verification Testing
- Compliance Verification Testing
- Authorization Testing
- Revision Testing
- Negative Testing
- Regression Testing

Property-Based Testing
End-to-End Testing

Testing shall evaluate both successful command execution and correct rejection of invalid commands.

43. NEGATIVE VERIFICATION TESTING

The system shall explicitly test invalid and unsafe command conditions.

Test conditions shall include:

Invalid Dimensions
Invalid Object Identifiers
Stale Revisions
Unauthorized Operations
Constraint Violations
Geometric Collisions
Invalid Regulatory Conditions
Missing Parameters
Ambiguous Commands
Unsupported Operations
Conflicting Redlines
Invalid State Transitions

The expected result shall be deterministic rejection, regeneration or review escalation according to the applicable policy.

44. SECURITY BOUNDARY

AI-generated content shall be treated as untrusted input until it has passed the applicable verification controls.

This includes:

Natural-Language Output
Structured Model Output
Generated Commands
Tool Arguments
Object Identifiers
Parameters
External Data References
Provider Responses

The AI model shall not receive implicit database authority.

Authority shall be granted only through validated OneDraft application operations.

45. ARCHITECTURAL DOCUMENTATION PROTECTION

Because OneDraft generates coordinated architectural documentation from the Project Model, a verified Project Model change may require corresponding documentation updates.

The system shall identify affected:

Plans

Elevations

Sections

Details

Dimensions

Annotations

Schedules

Code Documentation

Coordination Drawings

Document Packages

Documentation shall be generated from the resulting authoritative Project Model revision.

The drawing layer shall not independently establish a conflicting project state.

46. DRAWING CONSISTENCY

Following a validated Project Model mutation, the documentation sequence shall be:

MODEL REVISION → DOCUMENTATION REBUILD → SHEET COORDINATION → DRAWING VALIDATION

The system shall prevent obsolete drawing artifacts from being represented as the current coordinated documentation set.

All generated drawings shall remain traceable to the Project Model version from which they were generated.

47. SYSTEM INTEGRATION

The subsystem shall integrate with the following established OneDraft subsystems:

- 4.0 — API & Domain Schema Specification**
- 5.0 — Database Schema & OpenAPI Contract**
- 6.0 — Executable Foundation Implementation**
- 14.0 — Computational Project Model & Geometry Engine**
- 17.0 — Geometry & Parametric Model Engine**
- 18.0 — Compliance & Regulatory Engine**
- 19.0 — Validation & Verification Engine**
- 20.0 — Validation & Coordination Engine**
- 22.0 — Project Review, Redline, Acceptance & Approval Engine**
- 23.0 — Runtime Orchestrator**
- 24.0 — Job & Worker Execution System**
- 26.0 — Integration & Event Bus**
- 27.0 — Identity, Access & Security System**
- 28.0 — Identity, Security & Authorization System**
- 29.0 — Identity, Access Control & Security System**
- 30.0 — Observability, Telemetry & System Health**
- 31.0 — Testing, Quality Assurance & Verification System**
- 36.0 — Data Lifecycle, Retention & Provenance System**
- 37.0 — AI Intelligence & Model Provider Architecture**

This subsystem shall consume the capabilities established by these systems without redefining their primary responsibilities.

48. AI COMMAND LIFECYCLE

The complete AI command lifecycle shall be:

**GENERATED → RECEIVED → PARSED → VALIDATED → AUTHORIZED → VERIFIED
→ APPROVED FOR EXECUTION → EXECUTED → RECORDED**

Alternative lifecycle states shall include:

REJECTED
REQUIRES REVIEW
REGENERATING
DEFERRED
FAILED

No lifecycle transition shall bypass mandatory verification controls.

49. ARCHITECTURAL CONTROL MODEL

The completed architecture shall establish the following control relationship:

**CUSTOMER SPECIFICATION → ARIA → AI PROPOSAL → STRUCTURED COMMAND →
GUARDRAILS → AUTHORIZATION → CONSTRAINTS → GEOMETRY → COMPLIANCE
→ COORDINATION → EXECUTION POLICY → PROJECT MODEL → REVISION →
DOCUMENTATION**

The governing principle shall be:

ARIA GENERATES.

ONEDRAFT VERIFIES.

THE PROJECT MODEL RECORDS.

THE DOCUMENTATION ENGINE RENDERS.

THE CUSTOMER REVIEWS AND ACCEPTS.

The architecture shall preserve the Project Model as the authoritative representation of project state while permitting Aria to provide the intelligence required to interpret, generate and refine architectural operations.

50. STACK COMPLETION

The AI control architecture shall establish the controlled transition required to integrate Aria into the deterministic OneDraft application environment.

The completed architecture shall establish:

AI Intelligence

Model Provider Abstraction

AI Output Evaluation

Guardrails

Structured Commands

Authorization

Confidence Assessment

Risk Classification

Constraint Preservation

Geometry Verification

Compliance Verification

Coordination Verification

Deterministic Execution

Transactional Project Model Mutation

Verification Provenance

Auditability

Review Escalation

The resulting architecture is:

**AI INTELLIGENCE → AI VERIFICATION → AUTHORITATIVE PROJECT MODEL →
VALIDATED DOCUMENTATION**

The subsystem shall therefore provide the controlled transition between the AI architecture established in the preceding stack and the CAD, architectural graphics and interoperability architecture established in 39.0.

ARCHITECTURAL COMPLETION STATE

AI OUTPUT

↓

STRUCTURED COMMAND

↓

SCHEMA VALIDATION

↓

AUTHORIZATION

↓

PROJECT STATE VALIDATION

↓

CONSTRAINT VERIFICATION

↓

GEOMETRIC VERIFICATION

↓

REGULATORY VERIFICATION

↓

COORDINATION VERIFICATION

↓

CONFIDENCE / RISK POLICY

↓

DETERMINISTIC EXECUTION DECISION

↓

PROJECT MODEL REVISION

↓

DOCUMENTATION GENERATION

↓

REDLINE / REVIEW

The system shall never treat AI generation alone as authoritative project state.

MASTER SPECIFICATION STATEMENT

OneDraft establishes the AI Evaluation, Guardrails & Deterministic Command Verification architecture as the formal control boundary between Aria-generated intelligence and the authoritative OneDraft Project Model.

Aria shall provide the intelligence necessary to interpret customer specifications, reason over project context, generate architectural proposals, interpret redlines and construct structured project commands.

OneDraft shall independently verify those commands against project state, authorization, constraints, geometry, regulatory requirements, coordination requirements and execution policy before permitting them to become authoritative project state.

The resulting architecture is:

CUSTOMER SPECIFICATION → ARIA INTELLIGENCE → STRUCTURED COMMAND → DETERMINISTIC VERIFICATION → PROJECT MODEL → DOCUMENTATION

The Project Model remains the Single Source of Truth.

Aria remains the intelligence layer.

The verification architecture remains the control boundary.

The documentation engine remains the representation layer.

The customer remains the project-originating and review authority within the applicable OneDraft workflow.